

Technische Universität Wien

GPU Architecture & Computing

Semester Project

GPU-Accelerated Detection and Classification of Jigsaw Puzzle Pieces in Digital Images

Project Type: Group Project

Group Size: 5 Students

Topic Area: GPU Computing, Parallel Image Processing, Computer Vision

1. Project Overview

The goal of this project is to implement a GPU-accelerated system for the automatic detection and classification of canonical jigsaw puzzle pieces in digital images.

The project focuses on a complete image-processing pipeline for identifying puzzle pieces and classifying them according to their edge structure.

Students will design, implement, and evaluate a parallel version of this pipeline using GPU programming techniques. The focus is both on correctness and on understanding which parts of the algorithm benefit from GPU acceleration.

The final system should:

- detect all puzzle pieces in an image,
- extract their contours,
- analyse their geometric properties,
- classify each piece according to its edge types,
- and visualize the final classification result.

The project combines concepts from:

- GPU Computing,
- Parallel Algorithms,
- Computer Vision,
- Signal Processing,
- and Performance Optimization.

2. Problem Statement

Given an image containing several non-overlapping puzzle pieces, the system must:

1. preprocess the image,

2. separate foreground and background,
3. detect individual puzzle pieces,
4. extract the contour of each piece,
5. convert contours into one-dimensional signals,
6. detect corners, knobs, and holes,
7. classify the pieces,
8. and generate a labeled output image.

The puzzle pieces are assumed to be **canonical puzzle pieces**, meaning:

- every piece has four edges,
- each edge is either:
 - straight,
 - a knob,
 - or a hole,
- and opposite edges are approximately parallel.

3. Puzzle Piece Representation

Each puzzle piece is represented using a boundary description:

- L = straight edge,
- V = knob,
- C = hole.

Examples:

- LLCV
- CVCL
- VVCC

The system must normalize rotations and assign a unique class identifier to every piece.

4. Algorithm Overview

The complete algorithm consists of several stages.

4.1. Step 1 — Image Preprocessing

The input image is first converted to grayscale:

$$Y = 0.299R + 0.587G + 0.114B$$

The grayscale image is normalized to improve contrast and then thresholded to create a binary image.

Typical operations:

- RGB $\rightarrow$ grayscale conversion,
- normalization,
- thresholding,
- noise removal,
- morphological filtering.

GPU Parallelization:

- one thread per pixel,
- massively parallel image operations,
- shared memory optimization for filtering kernels.

4.2. Step 2 — Puzzle Piece Detection

The binary image is segmented into connected regions.

Each connected foreground region corresponds to one puzzle piece.

Small noisy regions must be removed.

Possible approaches:

- connected-component labeling,
- contour extraction,
- region growing.

GPU Parallelization:

- parallel connected-component algorithms,
- prefix-sum based region labeling,
- parallel region filtering.

4.3. Step 3 — Contour Extraction

For every puzzle piece, the outer contour must be extracted.

The contour is represented as an ordered list of boundary points:

$$P = \{p_0, p_1, p_2, \dots, p_n\}$$

The contour is traversed clockwise around the piece.

GPU Parallelization:

- parallel edge detection,
- contour point extraction,
- compact storage using prefix sums.

4.4. Step 4 — Contour-to-Signal Conversion

The contour is converted into a one-dimensional signal.

First, compute the center of the piece:

$$c = (x_c, y_c)$$

Then compute the distance from every contour point to the center:

$$s[i] = \sqrt{(x_i - x_c)^2 + (y_i - y_c)^2}$$

The resulting signal describes the shape of the puzzle piece.

Interpretation:

- corners produce strong peaks,
- knobs produce broad peaks,
- holes produce valleys.

GPU Parallelization:

- one thread per contour point,
- parallel distance computations,
- vectorized signal generation.

4.5. Step 5 — Signal Smoothing

The generated signal may contain noise and artifacts.

A moving average filter is applied:

$$\tilde{s}[i] = \frac{1}{k} \sum_{j=0}^{k-1} s[i+j]$$

Typical kernel sizes:

$$k = 5 \text{ or } 7$$

GPU Parallelization:

- sliding-window computations,
- shared-memory optimization,
- parallel convolution kernels.

4.6. Step 6 — Peak and Valley Detection

The smoothed signal is analyzed to detect:

- local maxima,
- local minima.

The system must identify:

- corners,
- knobs,
- holes.

Peak filtering uses:

- prominence,
- distance,
- threshold values.

GPU Parallelization:

- parallel local neighborhood comparisons,
- reduction operations,
- segmented scans.

4.7. Step 7 — Piece Classification

The four edges of the puzzle piece are classified.

Each edge receives one label:

$$L, \quad V, \quad C$$

The resulting four-character string defines the piece type.

Examples:

- CVCL
- VVCC
- LLCV

The final class is determined using a lookup table.

4.8. Step 8 — Visualization

The system overlays the original image with:

- bounding boxes,
- piece labels,
- detected contours.

Example output:

E_2 : VCCL

5. Suggested Division of Work

Student	Responsibilities
1	GPU image preprocessing: grayscale conversion, normalization, thresholding, noise filtering.
2	Puzzle piece segmentation: connected-component labeling, region extraction, noise removal.
3	Contour extraction and contour-to-signal conversion: distance computation, center estimation, signal generation.
4	Signal analysis: smoothing, peak detection, corner/knob/hole identification.
5	Classification, visualization, integration, benchmarking, performance evaluation, final report.

6. GPU Computing Aspects

The project should emphasize GPU-specific concepts:

- thread hierarchy,
- occupancy,
- memory coalescing,
- shared memory,
- synchronization,
- latency hiding,
- reduction operations,
- scan operations,

- workload balancing.

Students should compare:

- CPU execution time,
- GPU execution time,
- memory transfer overhead,
- scalability with image resolution.

7. Expected Deliverables

Each group should submit:

1. source code,
2. project report,
3. presentation slides,
4. benchmark results,
5. final output images,
6. documentation.

8. Evaluation Criteria

Criterion	Weight
Correctness of implementation	30%
GPU parallelization quality	10%
Performance improvements	10%
Code quality and Finale report	30%
Final presentation	20%

9. Possible Extensions

Groups interested in additional challenges may implement:

- adaptive thresholding,
- full puzzle reconstruction,
- CUDA stream optimizations,
- machine-learning based classification,
- real-time webcam input

10. References

1. OpenCV Documentation:
<https://opencv.org/>
2. CUDA Programming Guide:
<https://docs.nvidia.com/cuda/>
3. SciPy Signal Processing Documentation:
<https://docs.scipy.org/doc/scipy/reference/signal.html>